SuperFast: Fast Supernet Training Using Initial Knowledge

Moritz Thoma[1,3†*] Emad Aghajanzadeh[1,3†] Shambhavi Balamuthu Sampath[1,3]
Pierpaolo Mori[3] Nael Fasfous[3] Alexander Frickenstein[3] Manoj-Rohit Vemparala[3]
Daniel Mueller-Gritschneder[2] Ulf Schlichtmann[1]

[1]Technical University of Munich [2]TU Wien [3]BMW Group
† Equal Contribution. *Corresponding author moritz.thoma@tum.de.

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132779

***Abstract*—Once-for-all based neural architecture search (NAS) proposes to train a supernet once and extract specialized subnets from it for efficient deployment. This decoupling between training and search enables easy multi-target deployment without retraining. Nevertheless, the initial training cost has remained extremely high, with SOTA approaches like ElasticViT and NASViT taking more than 72 and 83 GPU days respectively. While other approaches have tried to accelerate the training by warming up the largest model in the search space, we argue that this is suboptimal, and knowledge is easier scaled upward than downward. Hence, we propose *SuperFast*, a simple, plug and play workflow, that (I.) pretrains a subnet of the supernet search space, and (II.) distributes its knowledge within the supernet before the training. SuperFast offers a substantial acceleration in the supernet training, resulting in a significantly better accuracy vs. training-cost trade-off. Using SuperFast on both ElasticViT and NASViT supernets achieves the baseline's accuracy 1.4× and 1.8× faster on the ImageNet dataset. Moreover, for a given time budget, SuperFast improves accuracy vs. latency trade-offs for subnets, gaining 4.0 p.p. for the 20-50 ms range on Pixel 6. Code available in https://github.com/MoritzTho/SuperFast.**

I. Introduction

The success of deep neural networks (DNN) models in various challenging tasks makes them attractive for use in real-world applications. However, the increasing growth in network's complexity and size makes their deployment challenging under mobile settings. On the other hand, bringing models to real world applications often requires deploying them on a wide range of different hardware devices. This is challenging as devices have different compute, bandwidth and power resources thereby resulting in different optimal architectures for different devices.

Neural architecture search (NAS) has explored many different ways to optimize the model architecture for any given hardware. However, as the number of different deployment target devices grows due to generational improvements, multiple performance tiers or switch of suppliers, so does the effort to find optimal architectures for each of them, growing the neural network design cost linearly with each new target (blue line in Fig. 1). To avoid a linear training cost increase with growing number of deployment targets, [1] introduced a once-for-all approach that creates one large supernet that is trained once and subsequently specialized for specific deployments, thereby decoupling the training and the search stage. Importantly, the single training mechanism for the supernet jointly optimizes all subarchitectures in the supernet, consequently not requiring finetuning of the specialized architectures extracted after the joint training finished. As a result, the training cost stays the same, even when the number of deployment targets grows (yellow line in Fig. 1).

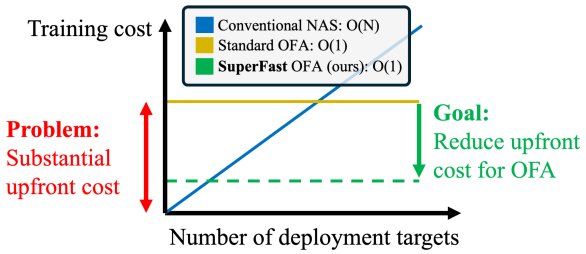

Fig. 1: For conventional NAS training cost increases linearly with the number of targets. Supernet NAS approaches reduce this to a single upfront training cost that pays off when targeting many devices. Our SuperFast method reduces this extremely high upfront cost, making supernet NAS more accessible.

Subsequent works have improved task performance by upgrading the search space architectures [5], [9], improving the sampling of architectures during joint training [9], [11] and improving the way knowledge is distributed between subnets during training [10], [11]. However, despite all these improvements, the training time has still remained a substantial concern in state-of-the-art taking more than 1750h [9], 2000h [5] on a NVIDIA A100 80G, making the training of supernet prohibitively expensive for most use cases. While some works have made attempts to reduce the training time of the supernet, they impose severe search space restrictions [7] or require warm up of the supernet [8], which we find is both inefficient and ineffective.

Differently, in this work we aim to reduce the expensive training time, using knowledge from a single, *smaller* architecture within the search space to infuse the entire search space with initial knowledge before starting the training. This simple, yet powerful approach allows for faster training and higher accuracies, thereby significantly reducing the GPU days required to achieve high performance.

The three main contributions of this paper can be summarized as follows:

- We propose SuperFast, the first method to make use of a small pretrained architecture to substantially accelerate the training of a large supernet search space.
- We show that using a small pretrained architecture together with our upscaling paradigm is more effective and time efficient than previous approaches relying on warmed up or fully trained supernets.
- Finally, we demonstrate the efficacy of our simple, yet effective SuperFast method on both ElasticViT and NASViT search spaces. We achieve a significant 1.4× and 1.8× time reduction while meeting the same accuracy as the baseline. We further show substantial accuracy improvements of up to 4.0 p.p. at a range of Pixel 6 latency budgets on the ImageNet dataset over the standard supernet training.

II. Related Work

A. Once-for-all Training

Early NAS methods have achieved significant advancements. However, their need to repeat search and training for each new hardware target makes them prohibitively costly in multi-target deployments. To address this issue, OFA [1] proposed a two-stage NAS to decouple training and search phases. OFA trains one large supernet once, and subsequently searches subnet architectures, that are specialized for different deployment targets. After the search, the specialized subnets only need minor fine tuning, leaving only one substantial training cost.

1) Updated Search Space: Followed by recent the success of vision transformer (ViT) models, hybrid CNN-ViT models have gained popularity for various vision tasks. NASViT [5] upgrades the OFA supernet architecture to such a SOTA hybrid CNN-ViT architecture, and for the first time, outperform prior pure CNN variants on mobile settings. Similarly, ElasticViT [9] uses a hybrid CNN-ViT supernet architecture but replaces the transformer blocks from NASViT with their own, more hardware friendly design. With their design and training method, they represent the current state-of-the-art for accuracy-latency trade-offs in supernets. Given the success of these hybrid architectures, we follow the same supernet search space and training recipes as NASViT and ElasticViT.

2) Training Cost Inefficiency: One challenge of supernet training is the vastness of the search space, including $O(10^{19})$ sub-architectures [1], which makes it challenging to optimize, resulting in prohibitive training cost. The work in [7] proposes to accelerate the optimization of the supernet by correlating different design decisions, reducing the search space to only 243 architectures. This achieves 2× reduction in training cost, but severely limits the flexibility of the resulting supernet, limiting its applicability for searching diverse, hardware specific subnets. BigNAS [11] proposes an in-place knowledge distillation to supervise sampled subnets with predictions made by the biggest model (supernet). This removes the need for further fine-tuning sampled subnets and reduce the training cost and is adopted by most subsequent works including NASViT and ElasticViT. Nevertheless, the finetuning done by OFA after the training is relatively inexpensive. Hence, the overall cost still remains high. InstaTune [8] is the closest to our work. They convert a *fully* trained architecture into a supernet by allowing elasticity to smaller variants of itself. Hence, they initialize their supernet search space with fully optimized weights of the *largest* architecture. Afterwards, they perform supernet training to jointly adopt it to downstream tasks. Differently, we argue that starting with the biggest network as initialization is suboptimal as scaling features down is harder than scaling features up. Hence, our SuperFast method starts with a *medium* sized subnet that *scales up* its knowledge to the supernet.

B. Knowledge Transfer

Knowledge transfer in neural networks seeks effective methods for transferring knowledge of a trained network to another. Knowledge distillation is one of its well-studied subfields to perform the transfer to smaller networks. Inversely, Net2Net [3] offers a rapid knowledge transfer to bigger networks based on network's function-preserving transformations. FNA [4] further improves these transformations, extends them to kernel dimension, and makes the transfer possible in both-directions. Inspired by these works, SuperFast applies an upscaling paradigm to distribute knowledge of a medium sized subnet, to a vast supernet search space. Table I provides a summary comparing our work to discussed related works.

TABLE I: Comparison of our SuperFast approach to related works for faster supernet optimization and knowledge transfer.

Works	Full Search Space	Avoids Supernet Warm Start	Knowledge Scaling
CompOFA [7]	✗	✓	✗
InstaTune [8]	✓	✗	✗
FNA [4]	-	-	✓
SuperFast (ours)	✓	✓	✓

III. Methodology

A. Supernet Background

We define supernet $\mathcal{M}$ with weights $\mathcal{W}$, and the set of containing architectures $\mathcal{A}$ (subnets). The set $\mathcal{A}$ is determined by definition of supernet's search space that creates the elasticity feature of it. Consider $\mathcal{M}$ with a set of micro-architectural blocks as B_i which can be either CNN- or ViT-based. Similar to the previous works, we consider three search dimensions, namely, width, depth, and kernel size. Width refers to the number of convolution channels for CNN and size of hidden feature in ViT blocks. Depth shows the number of layers in each CNN or ViT blocks, and kernel size determines the size of kernel used in CNN blocks. Therefore, each of these blocks is configurable by its depth, width, and kernel size, denoted as D_i, W_i and K_i, respectively.

The overall objective of supernet training can be defined as follows:

$$\min_{\mathcal{W}} \sum_{i \in \mathcal{A}} \mathcal{L}_{\text{val}}(\mathcal{W}_i) \tag{1}$$

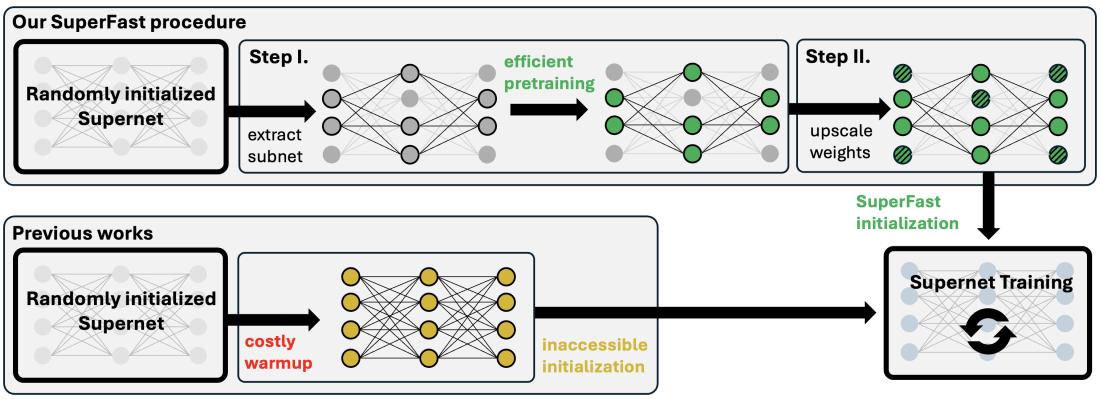

Fig. 2: Overview of our SuperFast procedure. Starting from a randomly initialized supernet, we extract and pretrain a subnet (step I.). Then, we upscale the subnet weights to the size of the supernet, achieving a better initialization more efficiently (step II.). Different to previous works, we do not rely on costly supernet warmup, or even complete supernet training, that may leave key features inaccessible to most subnets.

where $\mathcal{W}_i$ represents weights associated with subnet i and derived from the supernet weight $\mathcal{W}$. As all weights are derived from the single supernet weights $\mathcal{W}$, weights are shared within subnets of the supernet. Hence, supernet $\mathcal{M}$, which by definition is the biggest subnet in $\mathcal{A}$, contains all smaller subnet configurations in its weights $\mathcal{W}$.

To optimize the above objective, a small number of subnet samples (typically 4) are selected in each training iteration. Forward and backward passes are then performed to update the shared weights based on the resulting gradients. The sampling strategy typically follows the sandwich rule, selecting the largest and smallest subnets along with a few random ones in between. The largest subnet (supernet) is trained with labels, while the others use the supernet's predictions as soft labels. This training schema enables the co-optimization of multiple subnets simultaneously. However, it significantly increases the training cost compared to single-model training due to the need for multiple forward and backward steps in each iteration.

B. Primary Objective

The primary goal of this work is to reduce the training time in supernet NAS approaches T_{sup}. For that, we propose SuperFast a simple, plug and play workflow that leverages initial knowledge to accelerate the supernet training, resulting in shorter overall training time.

Specifically, SuperFast (I.) pretrains a single subnet $\mathcal{A}_{init}$ of the supernet $\mathcal{M}$ for time T_{init}, and (II.) distributes its knowledge within the supernet $\mathcal{M}$ before the training, resulting in an enhanced supernet $\mathcal{M}^*$ with a reduced training time T^*_{sup}. The relation is shown in Eq. 2 and the overall workflow is visualized in Fig. 2.

$$T_{\text{init}} + T^*_{\text{sup}} \ll T_{\text{sup}} \tag{2}$$

C. Subnet Selection for Pretraining

As for SuperFast's **first** stage, a subnet is selected from $\mathcal{A}$, denoted as $\mathcal{A}^{pre}_{init}$. The subnet is then trained standalone to achieve strong performance and acquire a sufficient amount of knowledge for effectively performing the task. We opt for pretraining a standalone network as it can acquire initial knowledge much quicker than a complex supernet. Specifically, training a medium sized subnet in the ElasticViT search space on ImageNet for one epoch is more than $8\times$ faster than training the corresponding supernet for one epoch, accelerating the initial knowledge gain substantially. To determine the best size of the network to pretrain, we study the weight sharing mechanism of supernets. There, the weight sharing is dependent on weight position rather than weight importance. Therefore, during pretraining, large networks may learn important features in weights that are inaccessible to most subnets. This problem is highlighted in Fig. 3, where a large subnet configuration has learned information in the upper channels during pretraining that cannot be accessed by smaller networks. Conversely, for smaller networks, the important features are learned in weights, that are shared between most subnets. Based on this insight, we further argue that choosing a small or medium sized subnet results in better knowledge transfer from a standalone, pretrained model to the supernet. This claim is controversial to the common supernet warming-up phase used in prior works such as in OFA [1], CompOFA [7], and NASViT [5].

D. Proposed Knowledge Distribution

The **second** stage leverages the knowledge from $\mathcal{A}^{pre}_{init}$ to provide a strong initialization for the joint optimization problem presented in Eq. 1. This is a challenging task that requires careful consideration, as weight-sharing and joint optimization could gradually diminish the knowledge previously acquired by the subnet. The key component of our approach is the effective knowledge transfer between the pretrained subnet and subnets in $\mathcal{A}$.

The knowledge transfer process varies depending on whether the target network is larger or smaller than $\mathcal{A}^{pre}_{init}$. Therefore, we address each case separately in our discussion: transferring knowledge to larger networks (upwards) and to smaller networks (downwards).

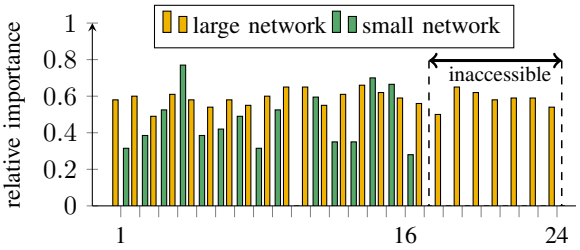

Fig. 3: Relative importance scores of channels within the same layer of two independently pretrained subnets of a supernet. The larger variant uses the 24 channel configuration of that layer and has important weights in the high channels 16-24, that will be inaccessible to smaller subnets that use the 16 channel configuration, degrading the transferability.

1) Downward Transfer: For downward transfer, we make use of the weight-sharing property. More specifically, we have made the following decisions across three search dimensions, i.e., kernel size, channel size, and network depth.

- **Kernel Cropping:** Larger kernels are center-cropped to match the desired size. For example, a 5x5 kernel is reduced to a 3x3 kernel by reusing the central part.
- **Channel Slicing:** The desired number of channels is selected by slicing from the beginning of the channel array. For instance, if 8 channels are needed from a 16-channel layer, the first 8 channels are selected.
- **Layer Utilization:** To reduce network depth, only the first few layers of the network are used. Specifically, the first L_i layers are utilized instead of the full set of N_i layers in block i.

2) Upward Transfer: In the upward transfer, the challenge lies in properly initializing additional untrained layers to ensure seamless integration with the pretrained weights. We explore two different approaches: (1) Random initialization and (2) Parameter remapping. While the random initialization initializes parameters that are not covered by the pretrained architecture randomly, the parameter remapping applies established techniques that can transfer knowledge from small to big networks. Specifically, it does:

- **Kernel Expansion:** Smaller kernels are placed at the center of larger kernels, with the remaining weights initialized to zero (Fig. 4a).
- **Width Expansion:** Additional channels are initialized with zero as introduced in previous work. There, this zero-initialization has been shown to integrate well with existing weights (Fig. 4b).
- **Depth Expansion:** Network depth is expanded by duplicating the last pretrained block. The weights from the last block in the pretrained network are copied into additional subsequent blocks in the larger network (Fig. 4c).

While these heuristics for initializing untrained weights have proven effective in FNA, as an ablation study, we also explore the effect of randomly initializing untrained weights. We compare the performance of these two settings to test the adaptability of untrained weights.

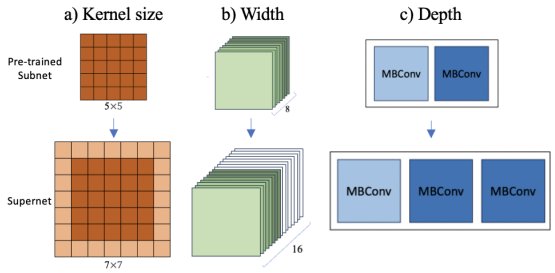

Fig. 4: Upward knowledge transfer across three search dimensions. For a) kernel size, the pretrained weights are zero padded. For b) width, additional channels are filled with zeros. For c) depth dimension, the last pretrained layer is repeated for all additional layers.

IV. Evaluation

A. Experimental Setup

We evaluate our SuperFast approach on ElasticViT [9] and NASViT [5] state-of-the-art search spaces on the ImageNet [6] dataset. For our experiments we build on the original code, training procedure and parameters as published on Github[1,2] at time of writing. Different to the original setup we reduce the number of GPUs used from 16 to 8 Nvidia V100 for ElasticViT and from 16 to 4 Nvidia A100 for NASViT for the execution of the experiments.

For pretraining specific submodels of the supernet, we follow the training parameters of [2] and stop the training after 50 epochs. All timing values are obtained by training a single epoch without validation on a single Nvidia A100 80GB GPU together with an AMD EPYC 7742 CPU.

B. Do Large Networks make for Better Initializers?

Past approaches have started their supernet training by warming up their supernet [1] or even relied on fully trained supernets [8]. Different to them we argue that using the biggest network as initialization for the supernet training is suboptimal as (1) it takes more time to train and (2) downscaling its features to smaller networks is difficult. Instead, in SuperFast we pretrain a medium sized architecture and use intelligent upscaling to preinitialize the search space more *efficiently* and *effectively*.

To verify this claim, we start by assessing the effectiveness of our intelligent upscaling method. For that, we initialize an ElasticViT search space with fully random weights (random-init), partially reloaded weights (direct-init) and our SuperFast intelligently initialized weights (SuperFast-init). We then train the supernet for one epoch to allow it to adjust itself to the loaded weights. The weights used for the initial knowledge are from a ElasticViT-M model, pretrained for 50 epochs. Table II shows the resulting average, minimum and maximum performance of the same 100 randomly sampled networks after 1 epoch of supernet training with the different initializations.

[1]https://github.com/facebookresearch/NASViT
[2]https://github.com/microsoft/Moonlit/tree/main/ElasticViT

Initialization	Top-1 (mean)	Top-1 (min)	Top-1 (max)
random-init	0.11	0.04	0.21
direct-init	7.68	**0.27**	53.55
SuperFast-init	**14.97**	0.24	**60.45**
supernet-init [8]	3.40	0.15	14.86

TABLE II: Performance of the same 100 random networks after 1 epoch of supernet training started from random-init, direct-init and SuperFast-init. The latter two are using knowledge from ElasticViT-M pretrained for 50 epochs. supernet-init uses a very large network for warm up, but falls short of ours.

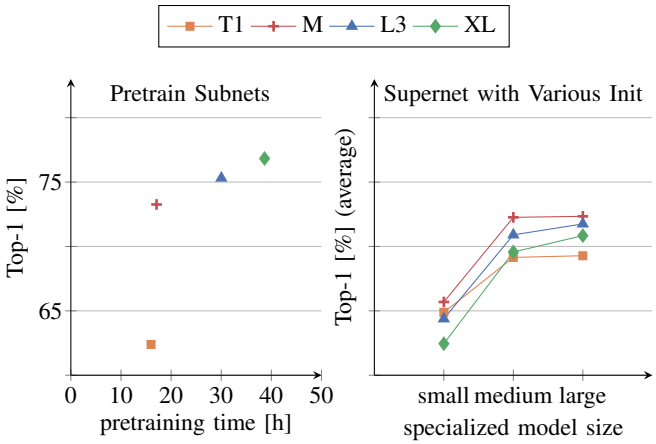

Fig. 5: Accuracy vs. training time trade-off when pretraining the T1, M, L3 and XL models of the ElasticViT search space for 50 epochs (left). Impact of the initialization on the average supernet performance for small, middle and large specialized networks after 10 epochs of supernet training (right). Very large networks take longer to pretrain and result in worse supernet performance, significantly reducing efficiency.

As we can see, random initialization barely moves beyond random predictions within the first epoch. Differently, the direct-init using the weights of the ElasticViT-M without any knowledge upscaling improves the performance significantly, resulting in a 7.57 p.p. increase of average top-1 and 53.34 p.p. increase of the best performing network over the fully random initialization. Our SuperFast-init improves beyond that: It surpasses the direct-init by 6.9 p.p. on the best performing network and by 7.29 p.p. on average, manifesting substantially better performance with our initialization. Finally, we compare with the initialization of the supernet of prior works [1], [8]. Specifically, we pretrain a ElasticViT-XL model, positioned on the top-end of the search space and use it to initialize the supernet. As can be seen in the table (supernet-init), using a very large network for the supernet initialization performs worse than just using the direct-init with ElasticViT-M, even without our upscaling.

Building on this finding, we move to investigate the impact of pretrained model size on the supernet training performance more carefully. For that, we pretrain architectures of four different sizes in different areas of the ElasticViT search

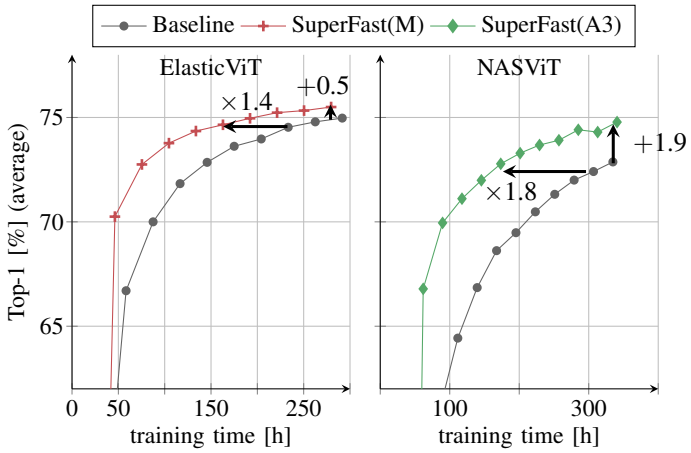

Fig. 6: Validation performance of the same 100 random architectures during the training of ElasticViT and NASViT supernets, with and without our SuperFast-init. In spite of the initial delay due to the pretraining, SuperFast achieves better accuracy vs training time trade-offs.

space. Specifically, we choose three predefined architectures ElasticViT-{T1, M, L3} at 62, 415, and 806 MFLOPs respectively. To properly represent the upper end of the search space, we again add the XL model, which has 1800 MFLOPs. Fig. 5 (left) shows the time cost and accuracy of all four models after 50 epochs of pretraining. As expected, both the accuracy and the training time of bigger models is higher. T1 is the only exception to this rule, as it does not saturate the compute of the GPU at the reference batch size. Notably, the M model achieves 95% of the accuracy of the XL model in less than half of the time, achieving a better accuracy vs. time trade-off.

Moving to the performance of the supernet using the different subnets as initialization, we find that the choice of subnet impacts the performance significantly. After 10 epochs of supernet training all supernet initializations result in relatively high performance already. In particular, we investigate the average top-1 accuracy of 100 random subnets that are grouped by their size into qualitative small, medium and large categories (Fig. 5, right). As can be seen there, the longest pretrained XL subnet with the highest accuracy does not result in the best performance. On the contrary, it falls behind the L3 and M subnets in all size categories, performing the worst of all of them for small subnets. The T1 initialized supernet is competitive to the M and L3 initialized supernet for small networks, but given its complexity is $\approx 20\times$ smaller than the largest sampled network, its ability of upscaling hits a limit. The medium sized M and L3 initialized supernets perform the best in most cases, striking a good balance between pretraining effort and upscaling ability. The fact that M initialized supernet outperforms the L3 initialized supernet which have similar pretraining performance, shows the remarkable efficiency of our approach and reinforces our hypothesis that scaling down features is generally harder than scaling up features.

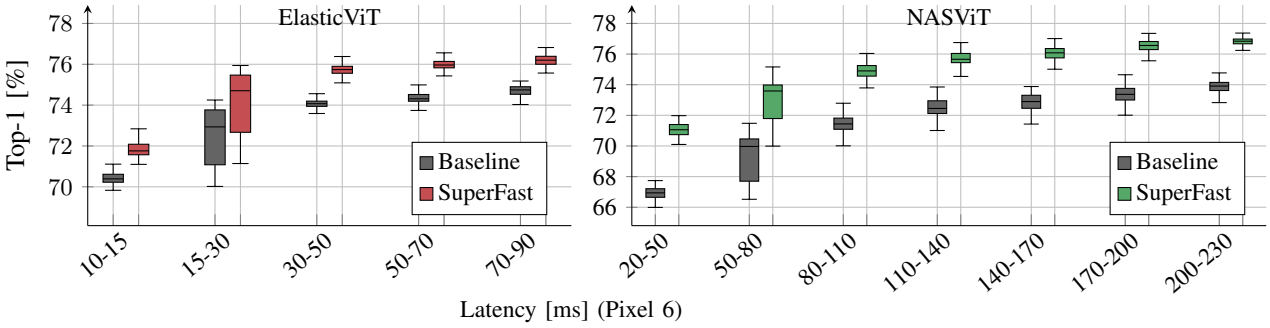

Fig. 7: Subnets sampled from ElasticViT and NASViT supernets, which were trained with- and without using our SuperFast procedure. As can be seen, our SuperFast procedure results in substantially higher accuracy for both search spaces on a wide range of latency budgets on Pixel 6.

C. Faster Supernet Training with SuperFast

After quantifying the advantages of using medium sized architectures with knowledge upscaling for initializing the supernet, we want to investigate the performance of our SuperFast approach on the ElasticViT and the NASViT search space. We start by investigating the time reduction and average accuracy gains achievable with our initialization. To evaluate both properties we train both supernets with the parameters in their public repositories for 1/6 of the total epochs. Afterwards, we repeat the same experiment with our SuperFast-init based on our pretrained ElasticViT-M subnet for the ElasticViT and a pretrained NASViT-A3 subnet for the NASViT supernet. NASViT-A3 was chosen as its position in the search space of NASViT is similar to the position of the M variant in the ElasticViT search space. Fig. 6 shows the accuracy over the time of training with and without our SuperFast-init for both search spaces. The plot accounts for the time it took to pretrain the subnets in both cases, adding a 33.91h and 17.1h offset for the SuperFast NASViT and ElasticViT training curves, respectively. In spite of the initial offsets, both SuperFast-init models catch up and overtake the baseline within the first 5 epochs after less than 50h and 60h for ElasticViT and NASViT respectively. During the course of the training, we observe substantial speedups as a result of our SuperFast-init. For ElasticViT we achieve the same accuracy $1.4\times$ earlier then the baseline. For NASViT the speedup is even greater, achieving a $1.8\times$ time reduction to reach the same accuracy. When using the same time budget for both, SuperFast improves the accuracy over the ElasticViT and NASViT baselines. After 250h and 350h the gain is 0.5 p.p. and 1.9 p.p., respectively with both seeing even higher gains when the time budget is shortened.

To further investigate if SuperFast supernet training can generate higher performing subnets across the entire search space beyond average improvements, we investigate subnets for multiple different latency budgets on the Google Pixel 6. We evaluate the baseline checkpoints of ElasticViT and NASViT after 146 h (50 epochs) and 279 h (50 epochs) of training and compare them to our checkpoints at 134 h (pretraining + 40 epochs) and 257 h (pretraining + 45 epochs) of training. We follow [9], to obtain accurate latency values for all subnets and divide the search space into 5 latency buckets for ElasticViT and 7 latency buckets for NASViT. For each bucket we sample 100 random networks and evaluate their accuracy. The results with and without SuperFast-init for both search spaces are shown in Fig. 7.

When evaluating our SuperFast-init results in Fig. 7, we see a clear improvement with SuperFast over both the NASViT and ElasticViT baselines for all latency budgets. In all but two cases, even the lowest quantile is above the highest quantile of the baseline, with the maximum distance between them being 2.3 p.p. When looking at the median values, the picture becomes even clearer with a minimum improvement of 1.4 p.p. and 2.9 p.p. and a maximum improvement of 1.8 p.p. and 4.0 p.p. for ElasticViT and NASViT, respectively. These performance improvements for all latency budgets demonstrate that SuperFast-init benefits *all* subnets in the search space. Looking beyond our own contribution, it is clear that the ElasticViT architecture design is very efficient, achieving *significantly* better accuracy latency trade-offs than its NASViT counterpart, therefore making it a much better choice for hardware deployment.

V. Conclusion

In this work, we introduced SuperFast, a plug-and-play procedure for supernets that accelerates the expensive supernet training, reducing its prohibitive cost. Different to previous works we do not warm up the largest network of the search space but find that scaling up features from small networks is easier than scaling down features from large ones. Consequently, for our approach, we made the choice to pretrain a medium sized model which is both more effective and more efficient. We demonstrated that our approach significantly improves supernet training time and accuracy on SOTA supernet search spaces, making it feasible to train them with limited resources. Future work may explore the usage of our method on other vision tasks, especially dense vision tasks, potentially demonstrating an even broader applicability of our method.

REFERENCES

[1] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. Once for all: Train one network and specialize it for efficient deployment. In *ICLR*, 2020.

[2] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. Efficientvit: Lightweight multi-scale attention for high-resolution dense prediction. In *ICCV*, pages 17302–17313, 2023.

[3] Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2net: Accelerating learning via knowledge transfer. *arXiv preprint arXiv:1511.05641*, 2015.

[4] Jiemin Fang, Yuzhu Sun, Kangjian Peng, Qian Zhang, Yuan Li, Wenyu Liu, and Xinggang Wang. Fast neural network adaptation via parameter remapping and architecture search. *arXiv preprint arXiv:2001.02525*, 2020.

[5] Chengyue Gong and Dilin Wang. Nasvit: Neural architecture search for efficient vision transformers with gradient conflict-aware supernet training. *ICLR*, 2022.

[6] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge. *Int. J. Comput. Vision*, 115(3):211–252, December 2015.

[7] Manas Sahni, Shreya Varshini, Alind Khare, and Alexey Tumanov. Compofa: Compound once-for-all networks for faster multi-platform deployment. *arXiv preprint arXiv:2104.12642*, 2021.

[8] Sharath Nittur Sridhar, Souvik Kundu, Sairam Sundaresan, Maciej Szankin, and Anthony Sarah. Instatune: Instantaneous neural architecture search during fine-tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1523–1527, 2023.

[9] Chen Tang, Li Lyna Zhang, Huiqiang Jiang, Jiahang Xu, Ting Cao, Quanlu Zhang, Yuqing Yang, Zhi Wang, and Mao Yang. Elasticvit: Conflict-aware supernet training for deploying fast vision transformer on diverse mobile devices. In *ICCV*, pages 5829–5840, 2023.

[10] Dilin Wang, Chengyue Gong, Meng Li, Qiang Liu, and Vikas Chandra. Alphanet: Improved training of supernets with alpha-divergence. In *International Conference on Machine Learning*, pages 10760–10771. PMLR, 2021.

[11] Jiahui Yu, Pengchong Jin, Hanxiao Liu, Gabriel Bender, Pieter-Jan Kindermans, Mingxing Tan, Thomas Huang, Xiaodan Song, Ruoming Pang, and Quoc Le. Bignas: Scaling up neural architecture search with big single-stage models. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part VII 16*, pages 702–717. Springer, 2020.